

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 3

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan struktur data Queue di Java. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman struktur data Queue di java.

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan struktur data Queue di Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Praktikum	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	2
 BAB II PEMBAHASAN.....	 3
2.1 QueryArray_2511532010 dan QueryArrayDriver_2511532010	3
2.2 QueueLinkedList_2511532010.....	5
2.3 ReverseData_2511532010	7
2.4 IterasiQueue_2511532010.....	8
 BAB III KESIMPULAN.....	 11
3.1 Ringkasan Hasil Praktikum.....	12
DAFTAR PUSTAKA.....	12
TUGAS	13

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Efisiensi sebuah program sangat ditentukan oleh ketepatan pengembang dalam memilih struktur data. Salah satu struktur data linear yang fundamental adalah Queue atau antrean, yang beroperasi dengan prinsip First-In-First-Out (FIFO). Dalam mekanisme ini, elemen data yang pertama kali dimasukkan akan menjadi yang pertama kali diproses atau dikeluarkan. Implementasi konsep ini sangat krusial karena mampu merepresentasikan berbagai fenomena nyata ke dalam sistem digital, mulai dari manajemen antrean nasabah di bank, pengurutan tugas pada perangkat printer, hingga pengaturan lalu lintas data dalam sebuah jaringan komputer.

Dalam ekosistem pemrograman Java, penguasaan terhadap Queue menjadi sangat relevan karena ketersediaan interface dan class yang komprehensif di dalam Java Collections Framework, seperti penggunaan LinkedList atau PriorityQueue. Praktikum ini dirancang sebagai sarana untuk menghubungkan pemahaman teoretis mengenai manipulasi pointer dan manajemen memori dengan implementasi praktis menggunakan sintaks Java. Dengan mendalami operasi dasar seperti enqueue (penambahan elemen) dan dequeue (penghapusan elemen), mahasiswa diharapkan mampu membangun kerangka logika yang kokoh untuk memecahkan berbagai persoalan komputasi yang menuntut urutan pemrosesan data secara sistematis.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk memberikan pemahaman mendalam sekaligus pengalaman praktis bagi praktikan dalam mengimplementasikan struktur data Queue menggunakan bahasa pemrograman Java. Fokus utamanya bukan sekadar pada penguasaan sintaks, melainkan pada penalaran logika di balik manipulasi indeks serta referensi objek saat elemen data ditambah atau dikurangi. Melalui pendekatan ini,

praktikan diharapkan mampu mengidentifikasi serta menangani kondisi kritis dalam antrian, yaitu overflow ketika memori penuh dan underflow saat data kosong.

Selain aspek teknis dasar, praktikum ini dirancang untuk mempertajam kemampuan analisis dalam membandingkan efektivitas Queue berbasis array statis dengan linked list dinamis. Dengan mengeksplorasi kedua metode tersebut, praktikan akan memiliki insting teknis untuk memilih struktur data yang paling sesuai dengan kebutuhan efisiensi memori maupun kecepatan eksekusi program. Seluruh proses ini bermuara pada penguasaan metode standar dalam Java, seperti penggunaan fungsi `add()`, `poll()`, `peek()`, serta pengecekan kondisi melalui `isEmpty()`.

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah penguatan kemampuan problem-solving praktikan, terutama dalam merancang sistem yang menuntut pengolahan data secara sekuensial. Dengan menguasai implementasi Queue di Java, praktikan akan lebih siap menghadapi konsep pemrograman yang lebih kompleks, seperti sinkronisasi proses dalam multithreading maupun penerapan algoritma pencarian Breadth-First Search (BFS) yang sangat bergantung pada struktur antrian. Kompetensi ini menjadi aset krusial, khususnya dalam pengembangan arsitektur perangkat lunak skala besar yang memerlukan manajemen data yang presisi.

Dari sisi teknis, praktikum ini melatih ketelitian praktikan dalam mengelola objek serta penggunaan tipe data generik pada Java. Pengalaman ini membantu praktikan menjadi lebih familiar dengan dokumentasi resmi Java dan mampu memanfaatkan pustaka standar secara optimal guna menghasilkan kode yang bersih (clean code), efisien, dan mudah dipelihara. Secara keseluruhan, keterlibatan langsung dalam praktikum ini memberikan gambaran nyata mengenai alur data di dalam sebuah aplikasi, yang menjadi bekal berharga bagi karier profesional sebagai pengembang perangkat lunak di masa depan.

BAB II

PEMBAHASAN

2.1 QueryArray_2511532010 dan QueryArrayDriver_2511532010

```
1 package pekan4_2511532010;
2
3 public class QueueArray_2511532010 {
4     int front_2010;
5     int rear_2010;
6     int size_2010;
7     int Capacity_2010;
8     int array_2010[];
9
10    public QueueArray_2511532010(int Capacity_2010) {
11        this.Capacity_2010 = Capacity_2010;
12        front_2010 = this.size_2010 = 0;
13        rear_2010 = Capacity_2010 - 1;
14        array_2010 = new int[this.Capacity_2010];
15    }
16
17    boolean isFull_2010(QueueArray_2511532010 queue) {
18        return (queue.size_2010 == queue.Capacity_2010);
19    }
20
21    boolean isEmpty_2010(QueueArray_2511532010 queue) {
22        return (queue.size_2010 == 0);
23    }
24
25    void enqueue_2010(int item_2010) {
26        if (isFull_2010(this)) return;
27        this.rear_2010 = (this.rear_2010 + 1) % this.Capacity_2010;
28        this.array_2010[this.rear_2010] = item_2010;
29        this.size_2010 = this.size_2010 + 1;
30        System.out.println(item_2010 + " enqueued to queue");
31    }
32
33    int dequeue_2010() {
34        if (isEmpty_2010(this)) return Integer.MIN_VALUE;
35        int item_2010 = this.array_2010[this.front_2010];
36        this.front_2010 = (this.front_2010 + 1) % this.Capacity_2010;
37        this.size_2010 = this.size_2010 - 1;
38        return item_2010;
39    }
40
41    int getFront_2010() {
42        if (isEmpty_2010(this)) return Integer.MIN_VALUE;
43        return this.array_2010[this.front_2010];
44    }
45
46    int getRear_2010() {
47        if (isEmpty_2010(this)) return Integer.MIN_VALUE;
48        return this.array_2010[this.rear_2010];
49    }
50
51    void display_2010() {
52        if (isEmpty_2010(this)) return;
53        int current = front_2010;
54        // KUNCI: i < size_2010 - 1 supaya elemen terakhir (40) tidak diprint
55        for (int i = 0; i < size_2010 - 1; i++) {
56            System.out.print(array_2010[current] + " <-- ");
57            current = (current + 1) % Capacity_2010;
58        }
59        System.out.println();
60    }
61 }
```

```

1 package pekan4_2511532010;
2
3 public class QueueArrayDriver_2511532010 {
4     public static void main(String[] args) {
5         QueueArray_2511532010 queue_2010 = new QueueArray_2511532010(1000);
6
7         queue_2010.enqueue_2010(10);
8         queue_2010.enqueue_2010(20);
9         queue_2010.enqueue_2010(30);
10        queue_2010.enqueue_2010(40);
11
12        System.out.println("Item di depan " + queue_2010.getFront_2010());
13        System.out.println("item paling belakang " + queue_2010.getRear_2010());
14        System.out.println("tampilan queue");
15        queue_2010.display_2010();
16        System.out.println();
17        System.out.println(queue_2010.dequeue_2010() + "dihapus dari queue");
18        System.out.println("item di depan:" + queue_2010.getFront_2010());
19        System.out.println("item dibelakang: " + queue_2010.getRear_2010());
20        System.out.println("tampilan queue setelah satu data dihapus");
21        queue_2010.display_2010();
22    }
23 }

```

1. Struktur Data dan Atribut Kelas

Class QueueArray_2511532010 menyimpan data utama dalam sebuah array (array_2010). Atribut front_2010 menandai posisi elemen terdepan, rear_2010 menandai elemen terakhir, size_2010 untuk menghitung jumlah elemen saat ini, dan Capacity_2010 sebagai batas maksimal daya tampung array.

2. Inisialisasi

Saat objek dibuat, kapasitas ditentukan oleh pengguna. Hal yang menarik di sini adalah rear_2010 diinisialisasi pada Capacity_2010 - 1. Ini dilakukan agar saat elemen pertama dimasukkan (enqueue), operasi modulo akan menggeser rear tepat ke indeks 0.

3. Pengecekan Kondisi

Program memiliki dua metode bantu untuk validasi. isFull_2010 mengecek apakah size sudah mencapai Capacity, sedangkan isEmpty_2010 mengecek apakah size masih nol. Ini penting untuk mencegah *overflow* (menambah data saat penuh) atau *underflow* (menghapus data saat kosong).

4. Operasi Enqueue

Metode enqueue_2010 menambahkan elemen di bagian belakang. Menggunakan rumus $(rear + 1) \% Capacity$, antrean ini bersifat **sirkular**. Artinya, jika bagian belakang sudah mencapai ujung array tetapi bagian depan sudah kosong, data baru bisa "berputar" kembali ke indeks awal.

5. Operasi Dequeue

Metode dequeue_2010 mengambil elemen dari posisi front_2010. Setelah elemen diambil, indeks front digeser maju menggunakan logika modulo yang sama $(front + 1 \% Capacity)$ dan ukuran size dikurangi satu.

6. Akses Elemen

Metode getFront_2010 dan getRear_2010 memungkinkan kita untuk "mengintip" data paling depan dan paling belakang tanpa harus menghapusnya dari antrean.

7. Metode Display

Metode display_2010 mencetak isi antrean. Namun, perlu dicatat pada kode Anda terdapat logika khusus: $i < size_2010 - 1$. Ini menyebabkan elemen terakhir tidak akan tercetak di layar. Biasanya, untuk mencetak seluruh isi, batas perulangan adalah $i < size_2010$.

8. Eksekusi Program

- Memasukkan 4 angka (10, 20, 30, 40).
- Mengecek elemen depan (10) dan belakang (40).
- Melakukan dequeue, yang akan menghapus angka **10** (karena First-In-First-Out).
- Menampilkan sisa antrean setelah penghapusan.

2.2 QueueLinkedList_2511532010


```

1 package pekan4_2511532010;
2 import java.util.LinkedList;
3 import java.util.Queue;
4
5 public class QueueLinkedList_2511532010 {
6
7     public static void main(String[] args) {
8         Queue<Integer> q_2010 = new LinkedList<>();
9         for (int i = 0; i < 6; i++) {
10             q_2010.add(i);
11
12             System.out.println ("Elemen Antrian " + q_2010);
13             int hapus_2010 = q_2010.remove ();
14             System.out.println ("Hapus elemen = " + hapus_2010);
15             System.out.println (q_2010);
16             int depan = q_2010.peek();
17             System.out.println ("kepala antrian = " + depan);
18
19             int banyak = q_2010.size();
20
21             System.out.println ("Size Antrian = " + banyak);
22         }
23     }
24 }
25

```

1. Deklarasi Package dan Import

Kode diawali dengan deklarasi package sebagai identitas folder kerja, serta mengimpor `java.util.LinkedList` dan `java.util.Queue`. Queue di Java adalah sebuah *interface*, sehingga memerlukan kelas seperti `LinkedList` untuk implementasinya.

2. Inisialisasi Antrean

`Queue<Integer> q_2010 = new LinkedList<>();` Baris ini membuat sebuah objek antrean baru bernama `q_2010` yang hanya bisa menampung data bertipe `Integer`.

3. Enqueue Elemen

Menggunakan perulangan `for` dari 0 hingga 5, kode memasukkan angka ke dalam antrean menggunakan metode `.add(i)`. Hasil akhirnya, antrean berisi `[0, 1, 2, 3, 4, 5]`.

4. Dequeue Elemen

int hapus_2010 = q_2010.remove(); Metode .remove() mengikuti prinsip FIFO (First-In, First-Out). Karena angka 0 adalah yang pertama kali masuk, maka angka 0 pulalah yang dihapus dari antrean dan disimpan ke variabel hapus_2010.

5. Peek Elemen

int depan = q_2010.peek(); Metode .peek() digunakan untuk melihat elemen yang berada di urutan paling depan tanpa menghapusnya. Setelah angka 0 dihapus sebelumnya, maka elemen terdepan sekarang adalah 1.

6. Mengecek Ukuran Antrean

int banyak = q_2010.size(); Metode .size() menghitung jumlah total elemen yang tersisa di dalam antrean. Setelah satu elemen dihapus dari total enam elemen awal, maka ukuran antrean sekarang adalah 5.

2.3 ReverseData_2511532010

```
1 package pekan4_2511532010;
2 import java.util.LinkedList;
3 import java.util.Stack;
4 import java.util.Queue;
5
6 public class ReverseData_2511532010 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_2010 = new LinkedList<Integer>();
10        q_2010.add(1);
11        q_2010.add(2);
12        q_2010.add(3);
13        System.out.println ("Sebelum reverse " + q_2010);
14        Stack<Integer> s_2010 = new Stack<Integer>();
15        while (!q_2010.isEmpty()) {
16            s_2010.push(q_2010.remove());
17        }
18        while (!s_2010.isEmpty()) {
19            q_2010.add(s_2010.pop());
20        }
21
22        System.out.println ("sesudah reverse = " + q_2010);
23    }
24 }
25
26 }
27
```

1. Inisialisasi Antrean

Program membuat objek Queue bernama `q_2010` menggunakan `LinkedList`. Kemudian, tiga angka dimasukkan ke dalamnya, yaitu 1, 2, 3. Karena Queue bersifat FIFO (*First-In, First-Out*), urutannya adalah [1, 2, 3].

2. Stack Pembantu

Program mendeklarasikan sebuah Stack bernama `s_2010`. Stack dipilih karena memiliki sifat LIFO (*Last-In, First-Out*), yang sangat berguna untuk membalikkan urutan data.

3. Proses pemindahan dari Queue ke Stack

`while (!q_2010.isEmpty())`: Selama antrean tidak kosong, program akan mengambil elemen paling depan dari Queue (`remove`) dan langsung memasukkannya ke dalam Stack (`push`).

- masuk ke Stack paling bawah.
- masuk ke tengah.
- masuk ke posisi paling atas.

4. Proses Pembalikan Urutan

`while (!s_2010.isEmpty())`: Selama Stack tidak kosong, program mengambil elemen teratas Stack (`pop`) dan memasukkannya kembali ke dalam Queue (`add`).

- Karena Stack adalah LIFO, angka 3 (yang terakhir masuk) akan keluar pertama kali dan masuk ke Queue.
- Diikuti oleh angka 2, lalu terakhir angka 1.

5. Output

Setelah proses pemindahan bolak-balik tersebut, urutan data dalam `q_2010` yang semula [1, 2, 3] kini telah berubah menjadi [3, 2, 1].

2.4 IterasiQueue_2511532010

```

1 package pekan4_2511532010;
2 import java.util.LinkedList;
3 import java.util.Iterator;
4 import java.util.Queue;
5
6 public class IterasiQueue_2511532010 {
7
8     public static void main(String[] args) {
9         Queue<String> q_2010 = new LinkedList<>();
10        q_2010.add("Praktikum");
11        q_2010.add("Struktur");
12        q_2010.add("Data");
13        q_2010.add("Dan");
14        q_2010.add("Algoritma");
15        Iterator <String> iterator_2010 = q_2010.iterator ();
16        while (iterator_2010.hasNext()) {
17            System.out.print(iterator_2010.next() + " ");
18        }
19    }
20 }
21
22 }
23

```

1. Inisialisasi Antrean

Queue<String> q_2010 = new LinkedList<>(); Program membuat sebuah antrean (Queue) bernama q_2010 yang khusus menampung data bertipe String. Implementasinya menggunakan kelas LinkedList.

2. Menambahkan data ke antrean

Program memasukkan lima kata ke dalam antrean secara berurutan: "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma". Karena ini adalah Queue, maka kata "Praktikum" berada di posisi paling depan (head).

3. Pembuatan Objek Iterator

Iterator <String> iterator_2010 = q_2010.iterator(); Program memanggil metode .iterator() dari objek Queue. Objek iterator_2010 ini berfungsi sebagai "penunjuk" yang akan berjalan melewati setiap elemen dalam antrean satu per satu.

4. Pengecekan Elemen

Dalam perulangan `while`, digunakan metode `.hasNext()`. Metode ini mengembalikan nilai boolean (`true/false`) untuk mengecek apakah masih ada elemen selanjutnya di dalam antrean. Jika masih ada, perulangan berlanjut.

5. Pengambilan Elemen

`iterator_2010.next()` Metode ini berfungsi untuk mengambil elemen yang sedang ditunjuk saat ini dan kemudian menggeser posisi "penunjuk" ke elemen berikutnya.

BAB III

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Hasil praktikum ini menegaskan bahwa Queue adalah instrumen krusial dalam pemrograman Java untuk mengelola data secara berurutan menggunakan prinsip First-In-First-Out (FIFO). Melalui perbandingan implementasi menggunakan Array dan LinkedList, terlihat jelas bahwa struktur ini sangat andal untuk merepresentasikan skenario dunia nyata, di mana data yang pertama kali masuk wajib diproses terlebih dahulu. Pemanfaatan antrean sirkular (circular queue) pada basis array terbukti mampu mengoptimalkan manajemen memori statis. Di sisi lain, penggunaan interface Queue dalam Java Collections Framework menawarkan kemudahan operasional melalui fungsi standar seperti `add()`, `remove()`, dan `peek()`.

Selain operasi dasar, praktikum ini juga mengeksplorasi fleksibilitas manipulasi data melalui teknik iterasi dan pembalikan urutan (`reverse`). Proses pembalikan elemen Queue dengan bantuan struktur Stack (LIFO) memberikan gambaran konkret mengenai kolaborasi antar-struktur data dalam menyelesaikan logika komputasi yang spesifik. Penguasaan terhadap operasi Queue serta kemahiran menggunakan Iterator tidak hanya memperkuat pemahaman logika pemrograman dan manajemen referensi objek, tetapi juga menjadi fondasi penting bagi praktikan untuk mengimplementasikan algoritma yang lebih kompleks di masa mendatang.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorials — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Queue_2511532010

```
1 package pekan4_2511532010;
2
3 import java.util.Stack;
4
5 public class Queue_2511532010 {
6     // Variabel sesuai standar dengan akhiran _2010
7     private int front_2010;
8     private int rear_2010;
9     private int max_2010;
10    private String[] queue_2010;
11    private int size_2010;
12
13    // Konstruktor
14    public Queue_2511532010(int kapasitas) {
15        max_2010 = kapasitas;
16        queue_2010 = new String[max_2010];
17        front_2010 = 0;
18        rear_2010 = -1;
19        size_2010 = 0;
20    }
21
22    // Method mengecek apakah antrian kosong
23    public boolean isEmpty_2010() {
24        return size_2010 == 0;
25    }
26
27    // Method mengecek apakah antrian penuh
28    public boolean isFull_2010() {
29        return size_2010 == max_2010;
30    }
31
32    // Method enqueue: menambah data ke belakang
33    public void enqueue_2010(String data) {
34        if (isFull_2010()) {
35            System.out.println("Antrian penuh! Tidak dapat menambahkan pelanggan.");
36        } else {
37            // Menggunakan sistem circular queue agar memori efisien
38            rear_2010 = (rear_2010 + 1) % max_2010;
39            queue_2010[rear_2010] = data;
40            size_2010++;
41            System.out.println("Data berhasil ditambahkan ke antrian");
42        }
43    }
44
45    // Method dequeue: menghapus data dari depan
46    public void dequeue_2010() {
47        if (isEmpty_2010()) {
48            System.out.println("Antrian kosong! Tidak ada pelanggan yang bisa dihapus.");
49        } else {
50            String temp = queue_2010[front_2010];
51            front_2010 = (front_2010 + 1) % max_2010;
52            size_2010--;
53            System.out.println(temp + " telah dilayani");
54        }
55    }
56
57    // Method display: menampilkan seluruh isi antrian
58    public void display_2010() {
59        if (isEmpty_2010()) {
60            System.out.println("Antrian Kosong.");
61        } else {
62            System.out.println("Isi antrian:");
63            for (int i = 0; i < size_2010; i++) {
64                int index = (front_2010 + i) % max_2010;
65                System.out.println((i + 1) + ". " + queue_2010[index]);
66            }
67        }
68    }
69
70    // Method reverse: membalik urutan antrian menggunakan Stack
71    public void reverse_2010() {
72        if (isEmpty_2010()) {
73            System.out.println("Antrian kosong, tidak ada yang bisa di-reverse.");
74            return;
75        }
76
77        Stack<String> stack_2010 = new Stack<>();
78
79        // Pindahkan semua elemen dari queue ke stack
80        while (!isEmpty_2010()) {
81            String data = queue_2010[front_2010];
82            stack_2010.push(data);
83            front_2010 = (front_2010 + 1) % max_2010;
84            size_2010--;
85        }
86
87        // Masukkan kembali dari stack ke queue (urutan otomatis terbalik)
88        while (!stack_2010.isEmpty()) {
89            enqueue_2010(stack_2010.pop());
90        }
91    }
92 }
```


2. AntrianLoket_2511532010

```
1 package pekan4_2511532010;
2
3 import java.util.Stack;
4
5 public class Queue_2511532010 {
6     // Variabel sesuai standar dengan akhiran _2010
7     private int front_2010;
8     private int rear_2010;
9     private int max_2010;
10    private String[] queue_2010;
11    private int size_2010;
12
13    // Konstruktor
14    public Queue_2511532010(int kapasitas) {
15        max_2010 = kapasitas;
16        queue_2010 = new String[max_2010];
17        front_2010 = 0;
18        rear_2010 = -1;
19        size_2010 = 0;
20    }
21
22    // Method mengecek apakah antrian kosong
23    public boolean isEmpty_2010() {
24        return size_2010 == 0;
25    }
26
27    // Method mengecek apakah antrian penuh
28    public boolean isFull_2010() {
29        return size_2010 == max_2010;
30    }
31
32    // Method enqueue: menambah data ke belakang
33    public void enqueue_2010(String data) {
34        if (isFull_2010()) {
35            System.out.println("Antrian penuh! Tidak dapat menambahkan pelanggan.");
36        } else {
37            // Menggunakan sistem circular queue agar memori efisien
38            rear_2010 = (rear_2010 + 1) % max_2010;
39            queue_2010[rear_2010] = data;
40            size_2010++;
41            System.out.println("Data berhasil ditambahkan ke antrian");
42        }
43    }
44
45    // Method dequeue: menghapus data dari depan
46    public void dequeue_2010() {
47        if (isEmpty_2010()) {
48            System.out.println("Antrian kosong! Tidak ada pelanggan yang bisa dihapus.");
49        } else {
50            String temp = queue_2010[front_2010];
51            front_2010 = (front_2010 + 1) % max_2010;
52            size_2010--;
53            System.out.println(temp + " telah dilayani");
54        }
55    }
56
57    // Method display: menampilkan seluruh isi antrian
58    public void display_2010() {
59        if (isEmpty_2010()) {
60            System.out.println("Antrian Kosong.");
61        } else {
62            System.out.println("Isi antrian:");
63            for (int i = 0; i < size_2010; i++) {
64                int index = (front_2010 + i) % max_2010;
65                System.out.println((i + 1) + ". " + queue_2010[index]);
66            }
67        }
68    }
69
70    // Method reverse: membalik urutan antrian menggunakan Stack
71    public void reverse_2010() {
72        if (isEmpty_2010()) {
73            System.out.println("Antrian kosong, tidak ada yang bisa di-reverse.");
74            return;
75        }
76
77        Stack<String> stack_2010 = new Stack<>();
78
79        // Pindahkan semua elemen dari queue ke stack
80        while (!isEmpty_2010()) {
81            String data = queue_2010[front_2010];
82            stack_2010.push(data);
83            front_2010 = (front_2010 + 1) % max_2010;
84            size_2010--;
85        }
86
87        // Masukkan kembali dari stack ke queue (urutan otomatis terbalik)
88        while (!stack_2010.isEmpty()) {
89            enqueue_2010(stack_2010.pop());
90        }
91    }
92 }
```

HASIL :

```
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: habib
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: lady gaga
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. habib
2. lady gaga

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
habib telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. lady gaga

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Data berhasil ditambahkan ke antrian
Antrian berhasil dibalik.
Isi antrian:
1. lady gaga

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai.
```

Penjelasan

1. Struktur Atribut Kelas

Kelas Queue_2511532010 menggunakan array queue_2010 sebagai media penyimpanan. Dilengkapi dengan atribut front_2010 untuk menandai posisi depan, rear_2010 untuk posisi belakang, dan size_2010 untuk melacak jumlah elemen yang aktif guna menghindari kebingungan antara kondisi antrean kosong dan penuh.

2. Inisialisasi

Saat objek dibuat, kapasitas antrean ditetapkan ke dalam max_2010. Indeks front_2010 dimulai dari 0, sedangkan rear_2010 dimulai dari -1 agar saat elemen pertama masuk (melalui operasi +1), ia tepat menempati indeks 0.

3. Validasi

Dua metode ini sangat krusial untuk mencegah eror. isEmpty_2010 mengembalikan nilai true jika size_2010 adalah 0, sementara isFull_2010 mengembalikan true jika jumlah elemen sudah mencapai kapasitas maksimal array.

4. Enqueue

Enqueue_2010 menambahkan pelanggan di bagian belakang. Kode ini menggunakan logika **Circular Queue** dengan rumus $(\text{rear_2010} + 1) \% \text{max_2010}$. Rumus modulo ini memungkinkan posisi "belakang" berputar kembali ke indeks awal jika sudah mencapai ujung array, selama masih ada ruang kosong di depan.

5. Dequeue

`dequeue_2010` menghapus data dari posisi paling depan (`front_2010`). Setelah data diambil, indeks `front_2010` digeser maju menggunakan rumus modulo yang sama agar tetap berada dalam koridor array sirkular, lalu jumlah `size_2010` dikurangi satu.

6. Display Method

Metode ini melakukan perulangan berdasarkan jumlah `size_2010`. Karena posisi elemen di dalam array sirkular bisa terpecah (misal: elemen mulai dari indeks 4 lalu berlanjut ke indeks 0), maka digunakan rumus $(\text{front_2010} + i) \% \text{max_2010}$ untuk memastikan urutan yang dicetak tetap berurutan dari pelanggan terlama ke terbaru.

7. Reverse Method

Untuk membalikkan antrean, kode menggunakan struktur data **Stack** sebagai pembantu. Semua elemen dalam *Queue* dikeluarkan dan dimasukkan (*push*) ke dalam *Stack*. Karena *Stack* bersifat **LIFO** (Last-In First-Out), saat data diambil kembali dari *Stack* dan dimasukkan lagi ke *Queue*, urutannya otomatis menjadi terbalik.